Delhi Metro Analysis

[PYTHON, PANDAS, SQL]

Sakiley Niharika | Data Science | 10-04-2026

1. Introduction

The Delhi Metro Rail Corporation (DMRC) operates one of the largest metro networks in India, providing fast, reliable, and eco-friendly transportation in Delhi.

This project focuses on analyzing Delhi Metro trip data using Python, Pandas, and SQL. The goal is to extract meaningful insights such as passenger trends, revenue generation, peak travel conditions, and station-wise performance.

PROBLEM AND IT'S APPROACH

Problem: Inconsistent and missing data in the dataset reduces accuracy.
Approach: Perform data cleaning using Pandas (handle nulls, remove duplicates, standardize formats).

Problem: Difficulty in extracting meaningful insights from raw data.
Approach: Use aggregation and grouping with SQL to summarize trends.

Problem: Large dataset causing slow performance.
Approach: Optimize queries using indexing

What is a Json file?

JSON is a **semi-structured data format**, it organizes data using key–value pairs.

Why It Is Difficult to Extract Data from Large Files

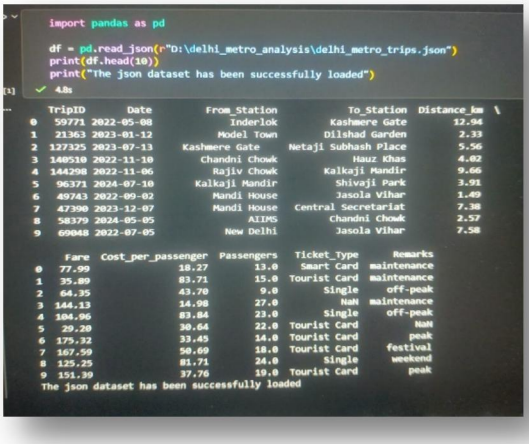
Extracting data from large files is inefficient because it requires high processing time, consumes significant memory, and involves complex parsing, which slows down performance and makes data handling more challenging.

How to load json using pandas?

1.import pandas
2.r = read mode
3.copy path of your json file and paste as shown: "your_path"

```
import pandas as pd

df = pd.read_json(r"D:\delhi_metro_analysis\delhi_metro_trips.json")
```



```
import pandas as pd

df = pd.read_json(r"D:\delhi_metro_analysis\delhi_metro_trips.json")
print(df.head(10))
print("The json dataset has been successfully loaded")
```

[1] ✓ 4.8s

```
   TripID       Date         From_Station           To_Station  Distance_km  \
0   59771 2022-05-08             Inderlok         Kashmere Gate        12.94
1   21363 2023-01-12            Model Town        Dilshad Garden         2.33
2  127325 2023-07-13     Kashmere Gate     Netaji Subhash Place         5.56
3  140510 2022-11-10        Chandni Chowk              Hauz Khas         4.02
4  144298 2022-11-06          Rajiv Chowk        Kalkaji Mandir         9.66
5   96371 2024-07-10        Kalkaji Mandir           Shivaji Park         3.91
6   49743 2022-09-02           Mandi House          Jasola Vihar         1.49
7   47390 2023-12-07           Mandi House   Central Secretariat         7.38
8   58379 2024-05-05                  AIIMS         Chandni Chowk         2.57
9   69048 2022-07-05             New Delhi          Jasola Vihar         7.58

     Fare  Cost_per_passenger  Passengers   Ticket_Type      Remarks
0   77.99                 18.27        13.0    Smart Card  maintenance
1   35.89                 83.71        15.0  Tourist Card  maintenance
2   64.35                 43.70         9.0        Single     off-peak
3  144.13                 14.98        27.0           NaN  maintenance
4  104.96                 83.84        23.0        Single     off-peak
5   29.20                 30.64        22.0  Tourist Card          NaN
6  175.32                 33.45        14.0  Tourist Card         peak
7  167.59                 50.60        18.0  Tourist Card     festival
8  125.25                 81.71        24.0        Single      weekend
9  151.39                 37.76        19.0  Tourist Card         peak
The json dataset has been successfully loaded
```

Revoming Extra Spaces in Station_Names using .Strip() method and Standardizing Station_Names

df["From_Station"] = df["From_Station"].str.strip().str.replace(r"\s+", " ", regex= True).str.upper()

r = raw string

\s= white space

+ = more than one

“ ” = replaces with single space

(Regex = True) Tells Pandas this is a **regular expression (pattern)**, not normal text

Str.upper() is used to capitalise letters in a word.

Handling Missing Values

print(df["Ticket_Type"].isnull().sum())

print(df["Ticket_Type"].unique())

df["Ticket_Type"] = df["Ticket_Type"].fillna(df["Ticket_Type"].mode()[o])

df["Passengers"] = df["Passengers"].fillna(df["Passengers"].median())

isnull().sum

This function checks **missing values** in your dataset, value is missing (NaN) ,.sum() counts the number of (NaN) values

What unique() does

It returns all unique (non-repeated) values from a column.

df["Ticket_Type"].mode()[0]
.mode()

Returns the **most frequent value(s)** in that column. Output is a **Series**, not a single value.

[0] Extracts the first mode value from the result.

fillna(...) Replaces missing values (NaN) with the given value.

.median() Returns the middle value of a numerical column when data is sorted. Works only on numeric data (int, float).

Ensuring Numeric Columns are Correctly Formatted

Converting Passengers from Float dtype to Int dtype

df["Passengers"] = df["Passengers"].round().astype(int)

round() Rounds decimal (float) values to the nearest whole number (by default).

.astype(int) Converts the data type to integer

Converting From JSON File To CSV File

Copy the cleaned data and use the keyword (to_csv) to convert into csv file

From JSON

#df = pd.read_json(r"D:\delhi_metro_analysis\delhi_metro_trips.json")

To Cleaned CSV

df_cleaned = df.copy()

df_cleaned.to_csv(r"D:\delhi_metro_analysis\delhi_metro_trips.csv", index=False)

Database Creation (SQL)

!pip install mysql-connector-python

Why you use it

You need this package to:

Connect Python to MySQL database , run SQL queries using Python, fetch data into Pandas

import mysql.connector
This line imports the MySQL connector library, which allows Python to communicate with a MySQL database.

conn = mysql.connector.connect(...)
This creates a connection to the MySQL server using:

host="localhost" → connects to your local machine

user="root" → uses the root username

password="sakiley" → uses your MySQL password

if conn.is_connected():This checks whether the connection to the database server is successful.

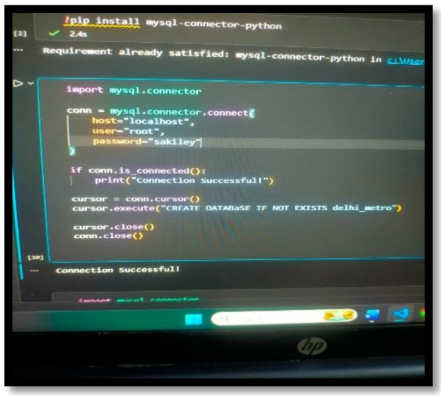
print("Connection Successful!"): If the connection works, it prints a confirmation message.

cursor = conn.cursor(): A cursor object is created. This is used to execute SQL commands.

cursor.execute("CREATE DATABASE IF NOT EXISTS delhi_metro")
This SQL command creates a database named **delhi_metro** only if it does not already exist.This prevents errors if the database is already created.

cursor.close() : Closes the cursor after executing the query.

conn.close(): Closes the connection to the MySQL server.



Creating a Structured table Schema

CREATE TABLE table_name (

column1 datatype ,

column2 datatype,

column3 datatype,

);

Importing cleaned data into the SQL table

data = list(df.itertuples(index=False, name=None)) data → A variable that will store all rows from the DataFrame

= → Assignment operator (stores value into data)

list(...) → Converts the output into a list format

Required because MySQL insertion works well with lists of rows

df → Pandas DataFrame (cleaned dataset)

.itertuples(...) → Converts each row of the DataFrame into a tuple, faster than looping with iterrows()

index=False → Excludes the index column from the output ,only actual data columns are taken

name=None → Removes tuple naming

Produces simple tuples like (val1, val2, ...) instead of named tuples

query = """

INSERT INTO metro_trips(

query → Variable storing the SQL command

""" → Multi-line string in Python

INSERT INTO → SQL command to add new records into a table

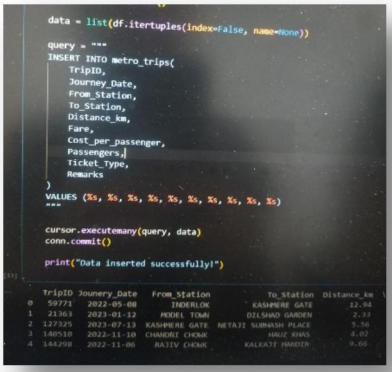
metro_trips → Name of the table where data will be inserted

VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)

VALUES → Specifies the data to insert

%s → Placeholder for actual values

- MySQL connector replaces %s with real data from tuples
- Prevents SQL injection and ensures safe insertion



```
data = list(df.itertuples(index=False, name=None))

query = """
INSERT INTO metro_trips(
    TripID,
    Journey_Date,
    From_Station,
    To_Station,
    Distance_km,
    Fare,
    Cost_per_passenger,
    Passengers,
    Ticket_Type,
    Remarks
)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
"""

cursor.executemany(query, data)
conn.commit()

print("Data inserted successfully!")
```

	TripID	Jounery_Date	From_Station	To_Station	Distance_km
0	59771	2022-05-08	INDERLOK	KASHMERE GATE	12.94
1	21363	2023-01-12	MODEL TOWN	DILSHAD GARDEN	2.33
2	127325	2023-07-13	KASHMERE GATE	NETAJI SUBHASH PLACE	5.56
3	140518	2022-11-10	CHANDNI CHOWK	HAUZ KHAS	4.02
4	144298	2022-11-06	RAJIV CHOWK	KALKAJI MANDIR	9.66

Creating Indexes for Faster Query Execution

An index in SQL is used to improve the speed of data retrieval from a table, especially when working with large datasets. Without an index, the database performs a full table scan, meaning it checks every row to find the required data, which can be time-consuming. By creating an index, the database uses a structured lookup mechanism (similar to an index in a book) to quickly locate the desired rows, significantly reducing query execution time. Indexes are particularly useful for columns frequently used in WHERE, JOIN, ORDER BY, or GROUP BY clauses. However, while indexes enhance read operations like SELECT, they can slightly slow down write operations such as INSERT, UPDATE, and DELETE, since the index also needs to be updated whenever the data changes.

```
cursor.execute("SHOW INDEX FROM metro_trips")
existing_indexes = [row[2] for row in cursor.fetchall()]

if "idx_route" not in existing_indexes:
    cursor.execute("CREATE INDEX idx_route ON metro_trips (From_Station, To_Station)")
if "idx_journey_date" not in existing_indexes:
    cursor.execute("CREATE INDEX idx_journey_date ON metro_trips (Journey_Date)")
if "idx_ticket" not in existing_indexes:
    cursor.execute("CREATE INDEX idx_ticket ON metro_trips (Ticket_Type)")

conn.commit()
```

This code retrieves the list of existing indexes from the metro_trips table using the SHOW INDEX command. It extracts the index names and stores them for comparison. Before creating any new index, it checks whether that index already exists in the table. This prevents the creation of duplicate indexes on the same columns. As a result, it reduces unnecessary storage usage and avoids additional overhead during data modification operations.

Data Analysis Using SQL

In this project, data is retrieved directly from the MySQL database using mysql.connector and loaded into a Pandas DataFrame (df) for easy handling. SQL queries are executed through the connector to select, filter, and aggregate data using clauses like SELECT, WHERE, GROUP BY, and functions such as COUNT, SUM, and AVG. ORDER BY is used to sort the results, and JOIN combines data from multiple tables when needed. Pattern-based filtering is done with LIKE, and DISTINCT ensures unique values. Using df allows us to manipulate, analyze, and clean the data efficiently before using it for further analysis or reporting.

SELECT Statement in SQL

The SELECT statement is used to retrieve data from one or more tables in a database. It allows you to choose **specific columns** or all columns using *. You can combine SELECT with other clauses like WHERE to filter rows, GROUP BY to aggregate data, and ORDER BY to sort the results.

WHERE Clause in SQL

The WHERE clause is used to **filter rows** in a table based on a specified condition. Only the rows that meet the condition are included in the result. It is often used with SELECT, UPDATE, or DELETE statements.

Use WHERE whenever you need **specific rows** from a table instead of retrieving all data. You can use comparison operators (=, >, <, >=, <=, !=) and logical operators (AND, OR, NOT) to create more complex conditions.

GROUP BY Clause

The GROUP BY clause is used to **group rows that have the same values** in one or more columns. It is often combined with aggregate functions like COUNT, SUM, AVG, MAX, and MIN to summarize data.

Use GROUP BY when you want **summary statistics** for categories, such as total passengers per ticket type or monthly revenue.

ORDER BY Clause

The ORDER BY clause is used to **sort query results** by one or more columns in ascending (ASC) or descending (DESC) order.

JOIN Clause

The JOIN clause is used to **combine rows from two or more tables** based on a related column. Common types include:

- INNER JOIN → returns only matching rows
- LEFT JOIN → returns all rows from the left table, matched rows from the right
- RIGHT JOIN → returns all rows from the right table, matched rows from the left

Use JOIN when your data is spread across **multiple tables** and you need to combine related information.

FROM Clause in SQL

The FROM clause specifies the **table or tables** from which to retrieve or manipulate data. Every SELECT, UPDATE, or DELETE query that works on table data must include FROM to tell the database where to look.

Use FROM when you need to **indicate the source table** for your query.

When using multiple tables, FROM can be combined with JOIN to connect them.

INSERT, UPDATE, and **DELETE** manage adding, modifying, or removing records.

LIKE operator helps find data based on patterns,

DISTINCT ensures only unique values are returned.

Using these SQL functions along with Python's Pandas DataFrame (df) allows efficient data retrieval, analysis, and preparation for reporting.

Passenger demand

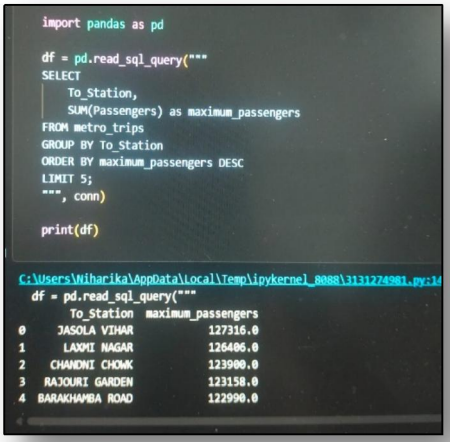
Passenger demand refers to the **number of passengers traveling on the metro network** over a given period. It shows how many people use each route, station, or ticket type, helping to identify **busy periods, popular routes, and trends in travel behavior**.

In analysis, passenger demand can be measured using:

- **Total passengers per day, month, or year**
- **Passengers by station** (which stations have the most entries/exits)
- **Passengers by ticket type** (e.g., Smart Card vs. Token)
- **Passengers by travel condition** (Peak, Off-Peak, Festival)

Understanding passenger demand helps in:

- **Planning train frequency** to reduce overcrowding
- **Resource allocation** like more staff or trains on busy routes
- **Revenue estimation** based on high-demand routes or periods



```
import pandas as pd

df = pd.read_sql_query("""
SELECT
    To_Station,
    SUM(Passengers) as maximum_passengers
FROM metro_trips
GROUP BY To_Station
ORDER BY maximum_passengers DESC
LIMIT 5;
""", conn)

print(df)
```

```
C:\Users\Niharika\AppData\Local\Temp\ipykernel_8088\3131274981.py:14
  df = pd.read_sql_query("""
        To_Station  maximum_passengers
0     JASOLA VIHAR            127316.0
1      LAXMI NAGAR            126406.0
2    CHANDNI CHOWK            123900.0
3   RAJOURI GARDEN            123158.0
4  BARAKHAMBA ROAD            122990.0
```

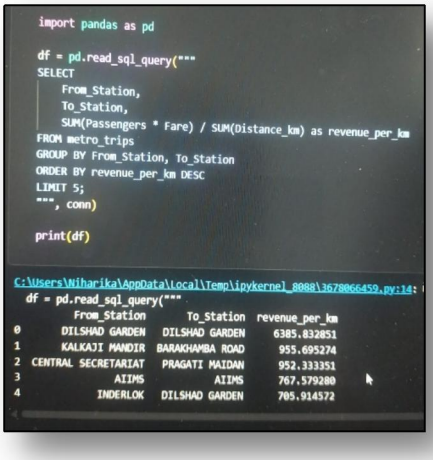
Revenue Trends

Revenue trends refer to the **patterns and changes in income generated** from passenger fares over time. By analyzing revenue trends, we can understand **which routes, ticket types, or time periods contribute most to income** and identify periods of high or low earnings.

Revenue can be analyzed by:

- **Daily, monthly, or yearly totals** to see how income changes over time
- **Ticket types** (e.g., Smart Card, Token) to determine the most profitable options
- **Travel conditions** (Peak, Off-Peak, Festival) to understand which periods generate more revenue
- **Routes** to identify the most profitable corridors
- This query shows which ticket type generates the highest revenue.

Analyzing revenue trends helps in **financial planning, resource allocation, and optimizing pricing strategies**.



```
import pandas as pd

df = pd.read_sql_query("""
SELECT
    From_Station,
    To_Station,
    SUM(Passengers * Fare) / SUM(Distance_km) as revenue_per_km
FROM metro_trips
GROUP BY From_Station, To_Station
ORDER BY revenue_per_km DESC
LIMIT 5;
""", conn)

print(df)
```

```
C:\Users\Wiharika\AppData\Local\Temp\ipykernel_8088\3678066459.py:14:
  df = pd.read_sql_query("""
         From_Station        To_Station  revenue_per_km
0       DILSHAD GARDEN   DILSHAD GARDEN     6385.832851
1       KALKAJI MANDIR  BARAKHAMBA ROAD      955.695274
2  CENTRAL SECRETARIAT   PRAGATI MAIDAN      952.333351
3                 AIIMS            AIIMS      767.579280
4             INDERLOK   DILSHAD GARDEN      705.914572
```

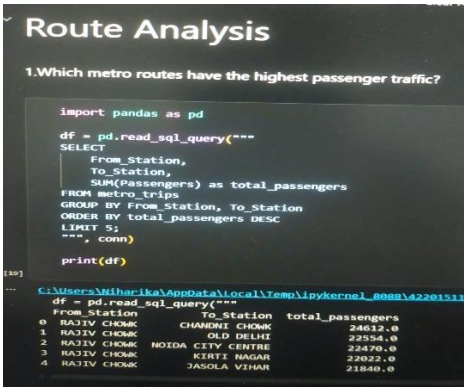
Route Analysis

Route popularity refers to how frequently passengers travel between specific stations or along certain corridors. It helps identify **the most used routes** in the metro network, showing which origin–destination pairs have the highest passenger counts.

Route popularity can be analyzed using:

- **Total passengers per route** (From_Station → To_Station)
- **Trips per time period** (daily, monthly, or peak hours)
- **Ticket type usage** on each route to understand passenger preferences
- This identifies the busiest routes and helps in planning train frequency and resources.

Route popularity is essential for **optimizing train schedules, managing congestion, and improving passenger experience**.



Route Analysis

1.Which metro routes have the highest passenger traffic?

```
import pandas as pd

df = pd.read_sql_query("""
SELECT
    From_Station,
    To_Station,
    SUM(Passengers) as total_passengers
FROM metro_trips
GROUP BY From_Station, To_Station
ORDER BY total_passengers DESC
LIMIT 5;
""", conn)

print(df)
```

```
C:\Users\Niharika\AppData\Local\Temp\ipykernel_8088\4220151
  df = pd.read_sql_query("""
  From_Station        To_Station  total_passengers
0  RAJIV CHOWK      CHANDNI CHOWK           24612.0
1  RAJIV CHOWK          OLD DELHI           22554.0
2  RAJIV CHOWK  NOIDA CITY CENTRE           22470.0
3  RAJIV CHOWK         KIRTI NAGAR           22022.0
4  RAJIV CHOWK        JASOLA VIHAR           21840.0
```

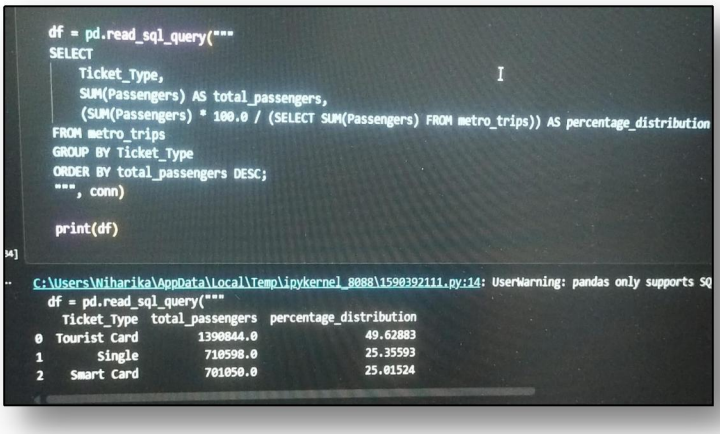
Ticket usage

Ticket usage refers to the distribution and frequency of different ticket types used by passengers in the metro system. It helps understand passenger preferences and which ticketing options are most popular, such as Smart Cards, Tokens, or Tourist Passes.

Ticket usage can be analyzed by:

- Counting the number of passengers per ticket type
- Comparing revenue generated from each ticket type
- Identifying trends during peak, off-peak, or festival periods

Analyzing ticket usage is important for planning ticketing strategies, optimizing revenue, and improving passenger convenience.



```
df = pd.read_sql_query("""
SELECT
    Ticket_Type,
    SUM(Passengers) AS total_passengers,
    (SUM(Passengers) * 100.0 / (SELECT SUM(Passengers) FROM metro_trips)) AS percentage_distribution
FROM metro_trips
GROUP BY Ticket_Type
ORDER BY total_passengers DESC;
""", conn)

print(df)
```

```
C:\Users\Niharika\AppData\Local\Temp\ipykernel_8088\1590392111.py:14: UserWarning: pandas only supports SQ
  df = pd.read_sql_query("""
   Ticket_Type  total_passengers  percentage_distribution
0  Tourist Card         1390844.0                49.62883
1       Single          710598.0                25.35593
2   Smart Card          701050.0                25.01524
```

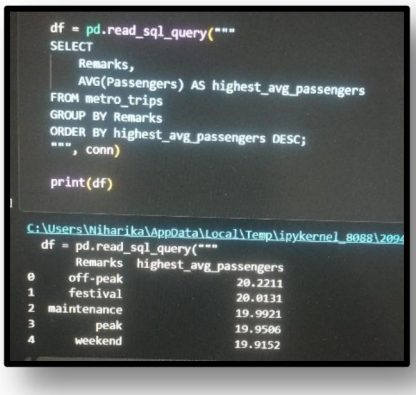
Travel Conditions

Travel conditions refer to the **different time-based or special situations** under which passengers travel, such as Peak hours, Off-Peak hours, Festival days, or Weekends. Analyzing travel conditions helps to understand **when the metro experiences the highest or lowest passenger traffic**.

Travel conditions can be analyzed by:

- **Counting passengers under each condition** (Peak, Off-Peak, Festival, Weekend)
- **Comparing revenue generated during different conditions**
- **Identifying which routes are busiest during specific conditions**

Analyzing travel conditions helps in **scheduling more trains during peak times, optimizing resource allocation, and improving passenger experience**.



```
df = pd.read_sql_query("""
SELECT
    Remarks,
    AVG(Passengers) AS highest_avg_passengers
FROM metro_trips
GROUP BY Remarks
ORDER BY highest_avg_passengers DESC;
""", conn)

print(df)
```

```
C:\Users\Niharika\AppData\Local\Temp\ipykernel_8088\2094
  df = pd.read_sql_query("""
       Remarks  highest_avg_passengers
0     off-peak                 20.2211
1     festival                 20.0131
2  maintenance                 19.9921
3         peak                 19.9506
4      weekend                 19.9152
```

Conclusion

In this project, I collected the Delhi Metro trip data and performed **data cleaning and preprocessing** using Python's Pandas library to ensure accuracy and consistency. Cleaned data was then **imported into a MySQL database** using mysql.connector, and SQL queries were used to retrieve, analyze, and summarize key information. I analyzed **Passenger Demand, Route Popularity, Ticket Usage, Travel Conditions, and Revenue Trends** to understand travel patterns and passenger behavior. Indexes were created on important columns to **improve query performance**, and data was carefully aggregated using functions like COUNT, SUM, AVG, and GROUP BY. By combining Pandas with SQL, I was able to efficiently manipulate, analyze, and visualize the data, leading to meaningful insights about metro usage and revenue patterns. This approach demonstrates a practical workflow for **data cleaning, database management, and analytical reporting**.